Projektarbeit

Fachbereich M – Master Maschienenbau Mechatraonik | WS 2025/2026

Entwicklung, Integration und Validierung einer ROS2-basierten Hinderniserkennung mit Reaktionsstrategie für automatisierte Modellfahrzeuge

Autoren:	Aly Elbermawy (5335848) Omar Elalami (5379642) Zakaria Elaboudi (5223183)
Betreuer:	Prof. Dr. Benedikt Lattke
Abgabedatum:	12. April 2026

Kurzfassung

Diese Projektarbeit beschreibt die Integration, Dokumentation und Bewertung eines scan-basierten *Follow-the-Gap*-Stacks (FTG) für das MXCarkit. Das Fahrzeug basiert auf einem NVIDIA-Jetson-System, arbeitet mit einem 2D-LiDAR und wird über eine Ackermann-Steuerkette mit VESC angesteuert. Ziel der Arbeit war es, einen robusten, nachvollziehbaren und sicher testbaren ROS 2-Datenpfad von `/scan` bis `/autonomous/ackermann_cmd` aufzubauen.

Die aktualisierte Zielarchitektur besteht aus vier logisch getrennten Stufen: (i) einer TF-basierten Vorverarbeitung des LiDAR-Scans, (ii) dem C++-Planerkern follow_the_gap_v0 im Scan-Modus, (iii) einem Python-Planer für Lenkwinkel- und Geschwindigkeitsvorgabe und (iv) einem Ackermann-Command-Knoten als Ausgabe in die bestehende MXCK-Steuerkette. Der bisherige Legacy-Pfad über obstacle substitution bleibt kompatibel im Repository, ist jedoch nicht mehr der primäre Laufzeitpfad.

Die theoretische Einordnung des Verfahrens erfolgt anhand der Originalarbeit von Sezer und Gokasan zum Follow-the-Gap-Verfahren, der CTU-Implementierung für F1/10-Fahrzeuge sowie der ROS 2/F1TENTH-Migrationsarbeit von Endler. Auf dieser Basis wird gezeigt, wie sich die klassische FTG-Idee (largest gap, gap center, final heading) in eine aktuelle ROS 2-Pipeline für ein Jetson-basiertes Lehr- und Versuchssystem übertragen lässt.

Inhaltsverzeichnis

Kurzfassung **II**

Inhaltsverzeichnis **III**

Abbildungsverzeichnis **V**

Tabellenverzeichnis **VI**

1 Grundlagen **1**

1.1 Aufgabenstellung und Ziel der Arbeit 1

1.2 Das MXCarkit als Zielplattform 2

1.3 Grundlagen von ROS 2 3

1.3.1 Workspace 3

1.3.2 Paket 4

1.3.3 Node 4

1.3.4 Launch-Dateien und Parameter 4

1.4 Follow-the-Gap als reaktiver Planungsansatz 5

1.4.1 Grundprinzip 5

1.4.2 Eigenschaften des Verfahrens 5

1.4.3 Scan-basierte Umsetzung im ROS 2-Kontext 5

2 Systembeschreibung und Durchführung **7**

2.1 Gesamtarchitektur des Systems 7

2.2 Algorithmische Grundlage 8

2.3 Stufe 1: LiDAR-Vorverarbeitung 9

2.3.1 Funktionale Aufgaben 9

2.3.2 Node Perception Erklärung 10

2.4 Stufe 2: Follow-the-Gap-Kern 10

2.4.1 Verarbeitungsschritte 10

2.4.2 Ausgabegrößen 10

2.5 Stufe 3: Planner 10

2.5.1 Funktionale Rolle 10

2.5.2 Interpretation der Sensordaten 11
2.6 Stufe 4: Control 11
2.6.1 Aufgabe 11
2.6.2 Systemintegration 11
2.7 Versuchsdurchführung 11
2.7.1 Versuchsaufbau 12
2.7.2 Parameter 13
2.7.3 Messmethodik 13

3 Diskussion und Bewertung 14
3.1 Bewertung der gewählten Gesamtarchitektur 14
3.1.1 Reduktion von Komplexität 14
3.1.2 Vergleich mit alternativen Ansätzen 14
3.2 Analyse der Planner- und Control-Schicht 15
3.3 Bewertung der experimentellen Ergebnisse 15
3.3.1 Quantitativer Vergleich der Validierungsläufe 15
3.3.2 Detailbewertung der Szenarien 15
3.3.3 Empfehlungen aus der Validierung 16
3.3.4 Robustheit gegenüber statischen Hindernissen 16
3.3.5 Verhalten bei dynamischen Änderungen 16
3.3.6 Grenzfälle 16
3.4 Einordnung in den wissenschaftlichen Kontext 17

4 Fazit und Ausblick 18
4.1 Fazit 18
4.2 Ausblick 18

Literaturverzeichnis 19

Literaturverzeichnis 19

Abbildungsverzeichnis

1.1 Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera. 2
1.2 Systemarchitektur und Hardware-Komponenten des MXCarkit [7]. . . . 3

2.1 Gesamtarchitektur des scan-basierten FTG-Stacks. Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die klare Trennung zwischen Wahrnehmung, Planung und Steuerung. 8
2.2 Schematische Darstellung des Follow-the-Gap-Prinzips. Erstellt unter Verwendung generativer KI-Tools. 9
2.3 Versuchsaufbau Aufnahme 1: Korridor mit einem statischen Hindernis. 12
2.4 Versuchsaufbau Aufnahme 2: Mehrere Hindernisse und dynamische Veränderung der Umgebung. 13

Tabellenverzeichnis

3.1 Vergleich der Metriken: Validierung Test 1 vs. Test 2 15

1 Grundlagen

Dieses Kapitel legt die fachlichen und systemtechnischen Grundlagen der vorliegenden Arbeit dar. Zunächst wird die Aufgabenstellung präzise eingeordnet und das Zielsystem beschrieben. Anschließend werden die relevanten Konzepte von ROS 2 erläutert, die für das Verständnis der implementierten Architektur notwendig sind. Abschließend wird der Follow-the-Gap-Algorithmus sowohl in seiner ursprünglichen Form als auch in der für diese Arbeit relevanten scan-basierten Ausprägung dargestellt.

1.1 Aufgabenstellung und Ziel der Arbeit

Ziel dieser Projektarbeit ist die Entwicklung, Integration und Validierung eines robusten, scan-basierten Hindernisvermeidungsverfahrens für ein automatisiertes Modellfahrzeug auf Basis von ROS 2.

Im Mittelpunkt steht die Realisierung eines durchgängigen und reproduzierbaren Datenpfads von der LiDAR-Sensorik bis zur fahrbaren Ackermann-Steuerung. Dabei wird nicht lediglich ein einzelner Algorithmus betrachtet, sondern die vollständige Verarbeitungskette bestehend aus Sensorvorverarbeitung, lokaler Planung und fahrzeugspezifischer Ansteuerung.

Die wesentlichen Teilziele der Arbeit sind:

- Aufbau eines konsistenten ROS 2-Datenpfads vom Rohsensor (`/scan`) bis zur Aktorik (`/autonomous/ackermann_cmd`),
- Integration des Follow-the-Gap-Algorithmus in eine reale Fahrzeugarchitektur,
- Gewährleistung reproduzierbarer Systemzustände durch klar definierte Launch- und Parameterkonfigurationen,
- experimentelle Validierung des Systemverhaltens anhand realer Fahrversuche,
- sowie die konsistente Dokumentation der tatsächlich implementierten Systemarchitektur im Repository `ftg_mxck`.

Im Unterschied zu rein theoretischen Arbeiten liegt der Fokus somit auf der praktischen Umsetzung innerhalb eines bestehenden Softwaresystems sowie dessen Interaktion mit realer Sensorik und Aktorik.

1.2 Das MXCarkit als Zielplattform

Das MXCarkit (MXCK) stellt die physische und softwareseitige Zielplattform dieser Arbeit dar. Es handelt sich um ein Jetson-basiertes Versuchsfahrzeug, das für Lehre und Entwicklung im Bereich autonomer Fahrfunktionen konzipiert ist.

Abbildung 1.1 zeigt den physischen Aufbau des Versuchsfahrzeugs mit den für diese Arbeit zentralen Sensorkomponenten.

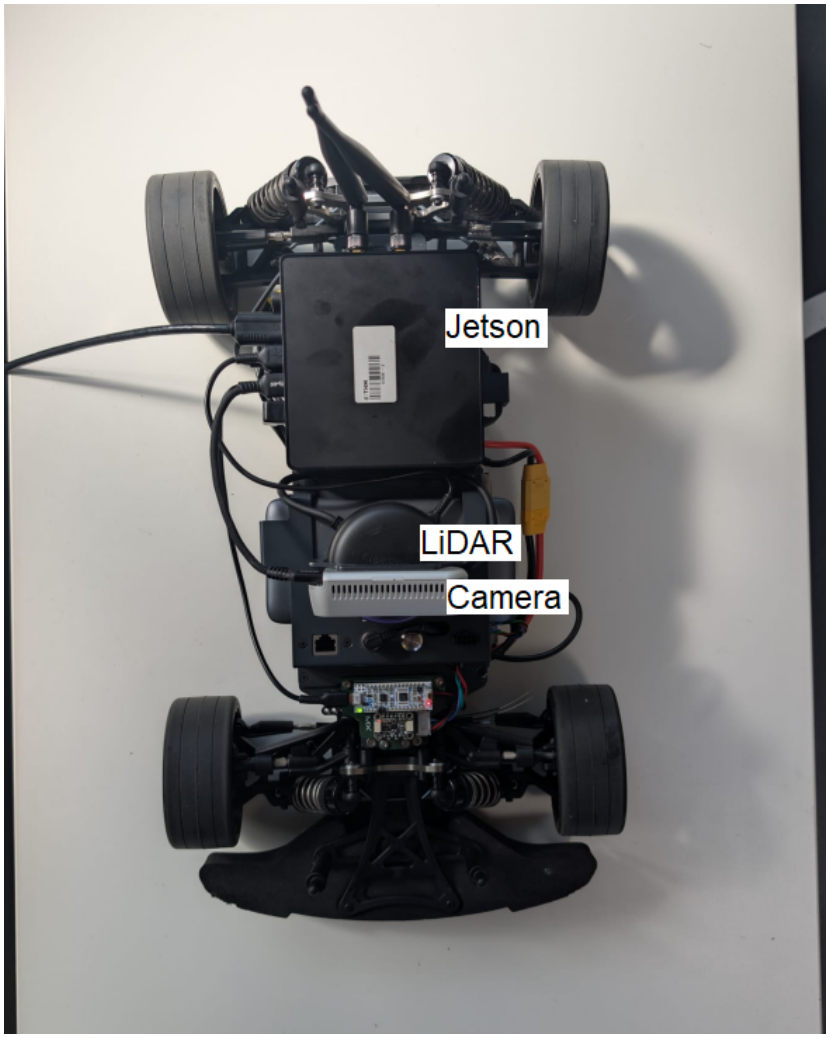

Abbildung 1.1: Draufsicht des MXCarkit mit Markierung der Hauptkomponenten: NVIDIA Jetson, LiDAR und Tiefenkamera.

Die Systemarchitektur des MXCK ist modular aufgebaut und nutzt eine containerisierte Umgebung. Eine detaillierte Übersicht der Hardware-Komponenten und deren Vernetzung ist in Abbildung 1.2 dargestellt.

Abbildung 1.2: Systemarchitektur und Hardware-Komponenten des MXCarkit [7].

Zentrale Funktionalitäten sind in separaten Workspaces organisiert, insbesondere:

- `mxck2_ws` für die Kernsysteme (Sensorik, Steuerung),
- `development_ws` für experimentelle Erweiterungen.

1.3 Grundlagen von ROS 2

Da die gesamte Systemarchitektur auf ROS 2 basiert, ist ein präzises Verständnis der zugrunde liegenden Konzepte erforderlich.

1.3.1 Workspace

Ein ROS 2-Workspace ist ein Verzeichnis, in dem mehrere Pakete gemeinsam verwaltet und gebaut werden. Die typische Struktur umfasst die Verzeichnisse `src/`, `build/`, `install/` und `log/`. Der Build-Prozess erfolgt üblicherweise über `colcon`.

Im Kontext des MXCK wird diese Struktur direkt genutzt, wobei unterschiedliche Workspaces verschiedene Verantwortlichkeiten abdecken.

1.3.2 Paket

Ein Paket stellt die zentrale organisatorische Einheit in ROS 2 dar. Es bündelt eine klar abgegrenzte Funktionalität, beispielsweise im Bereich Perception, Planung oder Steuerung.

Je nach Implementierungssprache unterscheiden sich die notwendigen Dateien:

- Python: `package.xml`, `setup.py`, `setup.cfg`,
- C++: `package.xml`, `CMakeLists.txt`.

Zusätzlich können Launch-Dateien und YAML-Konfigurationen Bestandteil eines Pakets sein.

1.3.3 Node

Ein Node ist ein eigenständiger Prozess innerhalb des ROS 2-Systems. Nodes kommunizieren über Topics, Services oder Actions und bilden gemeinsam einen verteilten Verarbeitungsgraphen.

Im vorliegenden System entspricht jede funktionale Stufe des FTG-Stacks einem eigenen Node, was eine klare Trennung der Verantwortlichkeiten ermöglicht.

1.3.4 Launch-Dateien und Parameter

ROS 2 verwendet bevorzugt Python-basierte Launch-Dateien zur Orchestrierung mehrerer Nodes. Diese definieren:

- welche Nodes gestartet werden,
- welche Parameter geladen werden,
- sowie die verwendeten Topic-Namen und Namespaces.

YAML-Dateien dienen dabei der strukturierten Parametrierung der Nodes. Für die Reproduzierbarkeit des Systems ist die Konsistenz zwischen Launch-Dateien, Parametern und tatsächlicher Node-Konfiguration entscheidend.

1.4 Follow-the-Gap als reaktiver Planungsansatz

Der Follow-the-Gap-Algorithmus (FTG) ist ein lokaler, reaktiver Planungsansatz zur Hindernisvermeidung. Im Gegensatz zu kartenbasierten Verfahren basiert FTG ausschließlich auf der aktuellen Sensorsicht und verzichtet auf eine globale Umgebungsrepräsentation.

1.4.1 Grundprinzip

Die grundlegende Idee des Verfahrens besteht darin, aus den erfassten Hindernissen freie Bereiche ("Gaps") zu bestimmen und die größte passierbare Lücke auszuwählen. Das Fahrzeug wird anschließend in Richtung des Zentrums dieser Lücke ausgerichtet.

Die klassische Formulierung umfasst drei Schritte:

1. Identifikation von Hindernissen und freien Bereichen,
2. Bestimmung eines geeigneten Gap-Zentrums,
3. Ableitung eines Fahrwinkels (Heading) basierend auf dieser Lücke.

Ein wesentliches Merkmal ist die rein lokale Entscheidungsfindung, die eine schnelle Reaktion auf dynamische Umgebungsänderungen ermöglicht.

1.4.2 Eigenschaften des Verfahrens

Das FTG-Verfahren zeichnet sich durch folgende Eigenschaften aus:

- geringe Anzahl an Parametern,
- robuste Reaktion in engen und unstrukturierten Umgebungen,
- keine Abhängigkeit von globaler Lokalisierung,
- Vermeidung klassischer lokaler Minima, wie sie bei Potentialfeldmethoden auftreten können.

Gleichzeitig ist das Verfahren inhärent lokal und trifft keine langfristigen Planungsentscheidungen.

1.4.3 Scan-basierte Umsetzung im ROS 2-Kontext

In praktischen ROS 2-Systemen wird FTG typischerweise direkt auf LiDAR-Daten angewendet. Die Eingangsgröße ist ein `sensor_msgs/LaserScan`, aus dem Hindernisabstände und freie Bereiche extrahiert werden.

Im Rahmen dieser Arbeit wird der Algorithmus nicht isoliert verwendet, sondern in eine mehrstufige Pipeline eingebettet. Dabei erfolgt eine explizite Vorverarbeitung des Scans, bevor dieser dem eigentlichen FTG-Kern übergeben wird.

Der resultierende Datenfluss folgt konzeptionell der Struktur:

```
/scan
  -> Vorverarbeitung (Filterung, Transformation)
  -> FTG-Kern (Heading-Bestimmung)
  -> Planner (Policy und Geschwindigkeit)
  -> Control (Ackermann-Ausgabe)
```

Diese Aufteilung ermöglicht es, den algorithmischen Kern von fahrzeugspezifischen Anforderungen zu entkoppeln und gleichzeitig eine hohe Transparenz im Gesamtsystem zu gewährleisten.

2 Systembeschreibung und Durchführung

Dieses Kapitel beschreibt die konkret umgesetzte Systemarchitektur sowie die methodische Durchführung der Implementierung und Validierung des Follow-the-Gap-Stacks im Repository `ftg_mxck`. Der Fokus liegt auf dem scan-basierten Hauptpfad, der als primäre Laufzeitarchitektur definiert wurde.

2.1 Gesamtarchitektur des Systems

Die entwickelte Pipeline folgt einer strikt modularen Struktur, die sich in vier logisch getrennte Verarbeitungsebenen gliedert:

1. LiDAR-Vorverarbeitung (Perception)
2. Reaktiver Planerkern (Follow-the-Gap)
3. Policy-basierte Entscheidungslogik (Planner)
4. Fahrzeugschnittstelle (Control)

Die resultierende Datenflusskette ist in Abbildung 2.1 dargestellt.

```
/scan (LaserScan, frame: laser)
  |
  ▼
┌──────────────────────┐
│  scan_preprocessor_node │  Paket: mxck_ftg_perception
│  TF: base_link ← laser  │  Rezentrierung auf Fahrzeug-Front
│  FOV: ±60° (120°)       │  Clipping: 0.25–10.0 m
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/scan_filtered  (LaserScan, frame: base_link)
         └──▶ /autonomous/ftg/front_clearance (Float32, Meter)
               |                                  |
               ▼                                  |
┌──────────────────────┐                          |
│    follow_the_gap_v0    │  Paket: follow_the_gap_v0│
│    (C++, input_mode=scan)│  CTU FTG-Algorithmus     |
└──────────────────────┘                          |
         |                                        |
         |                                        |
         ├──▶ /final_heading_angle (Float32, rad)    |
         ├──▶ /gap_found           (Bool)            |
         ├──▶ /visualize_obstacles (Marker)          |
         ├──▶ /visualize_largest_gap (PointStamped)  |
         └──▶ /visualize_final_heading_angle (PoseStamped)
               |                                  |
               ▼                                  ▼
┌──────────────────────────────────────────────┐
│              ftg_planner_node                  |
│  Paket: mxck_ftg_planner                       |
│  Speed-Policy: clearance + steering → Geschwindigkeit|
│  Heading-Smoothing + Speed-Smoothing           |
└──────────────────────────────────────────────┘
         |
         |
         ├──▶ /autonomous/ftg/gap_angle      (Float32, rad)
         ├──▶ /autonomous/ftg/target_speed   (Float32, m/s)
         └──▶ /autonomous/ftg/planner_status (String)
               |
               ▼
┌──────────────────────┐
│    ftg_command_node     │  Paket: mxck_ftg_control
│    → AckermannDriveStamped
└──────────────────────┘
         |
         |
         ├──▶ /autonomous/ackermann_cmd       (AckermannDriveStamped)
         └──▶ /autonomous/ftg/control_status  (String)
               |
               ▼
         [vehicle_control]  (extern, nicht Teil dieses Stacks)
```

Abbildung 2.1: Gesamtarchitektur des scan-basierten FTG-Stacks. Die Pipeline reicht vom Rohscan bis zur Ackermann-Ausgabe und zeigt die klare Trennung zwischen Wahrnehmung, Planung und Steuerung.

Der vollständige Aufbau sowie alle Konfigurationsdateien sind im Repository `ftg_-mxck` dokumentiert. Im Rahmen dieser Arbeit wird auf eine vollständige Reproduktion aller Startbefehle verzichtet; stattdessen wird die Architektur konzeptionell beschrieben und auf die implementierten Komponenten verwiesen.

2.2 Algorithmische Grundlage

Der eingesetzte Planungsansatz basiert auf dem Follow-the-Gap-Verfahren nach Sezer und Gokasan [1]. Die konkrete Implementierung orientiert sich an der F1TENTH-/CTU-Referenz [3] und wurde unverändert als C++-Kern in den Stack integriert.

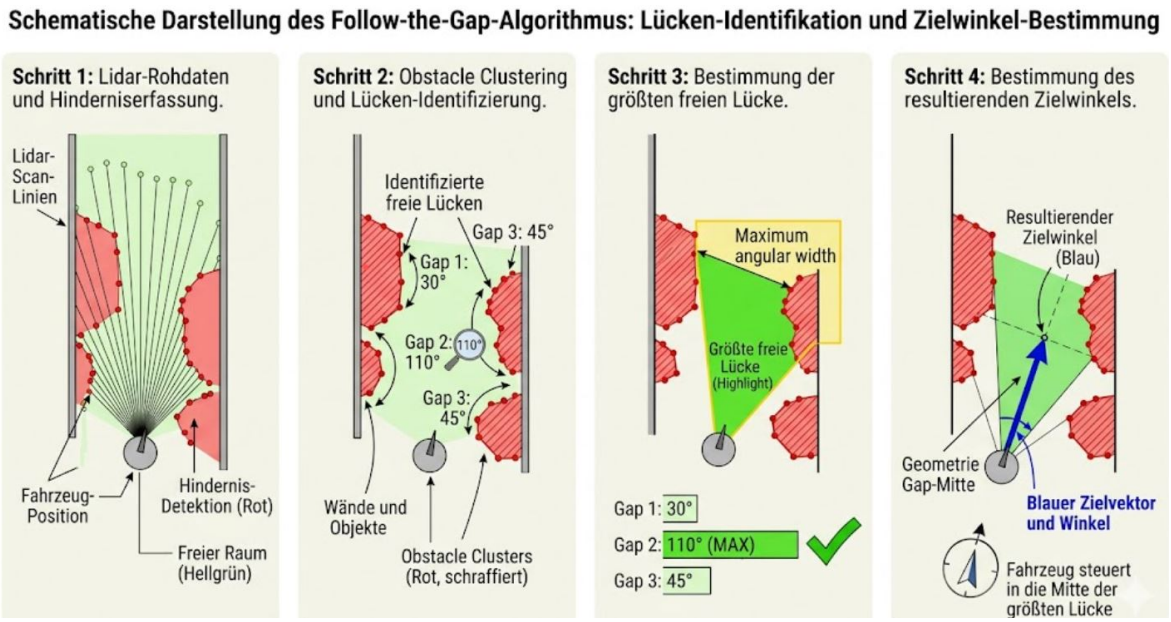

Abbildung 2.2: Schematische Darstellung des Follow-the-Gap-Prinzips. Erstellt unter Verwendung generativer KI-Tools.

Die zentrale Idee besteht darin, aus einem eindimensionalen Distanzprofil die größte kollisionsfreie Lücke zu bestimmen und das Fahrzeug entlang deren Mittelpunkt auszurichten. Diese Strategie ermöglicht eine robuste reaktive Navigation ohne globale Karteninformation.

2.3 Stufe 1: LiDAR-Vorverarbeitung

Die Vorverarbeitung transformiert den Rohscan in eine für den FTG-Kern geeignete Repräsentation.

2.3.1 Funktionale Aufgaben

Die implementierte Verarbeitung umfasst:

- Transformation des Scans in den Fahrzeugrahmen mittels TF,
- Definition eines frontalen Sichtfensters,
- Entfernung ungültiger Messwerte,
- Glättung des Signals zur Rauschreduktion,
- Berechnung des minimalen Frontabstands.

Diese Schritte sind notwendig, um die geometrische Konsistenz zwischen Sensor und Fahrzeug sicherzustellen.

2.3.2 Node Perception Erklärung

Bla Bla Bla

2.4 Stufe 2: Follow-the-Gap-Kern

Der Planerkern bildet das algorithmische Zentrum des Systems. Er verarbeitet den vorbereiteten Scan und berechnet daraus einen Richtungswinkel.

2.4.1 Verarbeitungsschritte

Innerhalb des Algorithmus erfolgen:

1. Identifikation von Hindernisbereichen,
2. Bestimmung zusammenhängender freier Bereiche (Gaps),
3. Auswahl des größten befahrbaren Gaps,
4. Berechnung eines Zielwinkels innerhalb dieses Gaps.

2.4.2 Ausgabegrößen

Der Algorithmus liefert:

- einen kontinuierlichen Heading-Winkel,
- ein binäres Signal zur Gap-Erkennung.

Diese Ausgabe ist bewusst fahrzeugagnostisch gehalten und enthält keine Informationen über Geschwindigkeit oder Dynamik.

2.5 Stufe 3: Planner

Der Planner interpretiert die algorithmische Ausgabe im Kontext der Fahrzeugdynamik und Sicherheitsanforderungen.

2.5.1 Funktionale Rolle

Im Gegensatz zum FTG-Kern handelt es sich hierbei nicht um einen Planungsalgorithmus im engeren Sinne, sondern um eine Entscheidungslogik, die folgende Aufgaben übernimmt:

- Validierung der Eingangsdaten (Freshness),

- Begrenzung des Lenkwinkels,
- adaptive Geschwindigkeitswahl in Abhängigkeit vom Frontabstand,
- Zustandsbewertung des Systems.

2.5.2 Interpretation der Sensordaten

Die Geschwindigkeit wird als Funktion des minimalen Abstands gewählt. Dabei gilt:

- große Abstände → hohe Geschwindigkeit,
- kleine Abstände → reduzierte Geschwindigkeit,
- kritische Abstände → Stopp.

Diese Policy stellt sicher, dass der reaktive Charakter des FTG-Ansatzes nicht zu unsicheren Fahrzuständen führt.

2.6 Stufe 4: Control

Die letzte Stufe des Systems übernimmt die Transformation der Planner-Ausgabe in fahrzeugspezifische Steuerbefehle.

2.6.1 Aufgabe

Die berechneten Sollwerte werden in ein Ackermann-kompatibles Nachrichtenformat überführt und an die bestehende Fahrzeugsteuerung übergeben.

2.6.2 Systemintegration

Ein wesentliches Designkriterium ist, dass die bestehende Sicherheits- und Steuerlogik des MXCarkit unverändert bleibt. Der entwickelte Stack liefert ausschließlich Sollwerte und greift nicht direkt in die Aktorik ein.

2.7 Versuchsdurchführung

Die Validierung des Systems erfolgte anhand realer Fahrversuche im Innenraum.

2.7.1 Versuchsaufbau

Die Tests wurden in einem strukturierten Korridor durchgeführt, der durch gezielt platzierte Hindernisse definiert wurde.

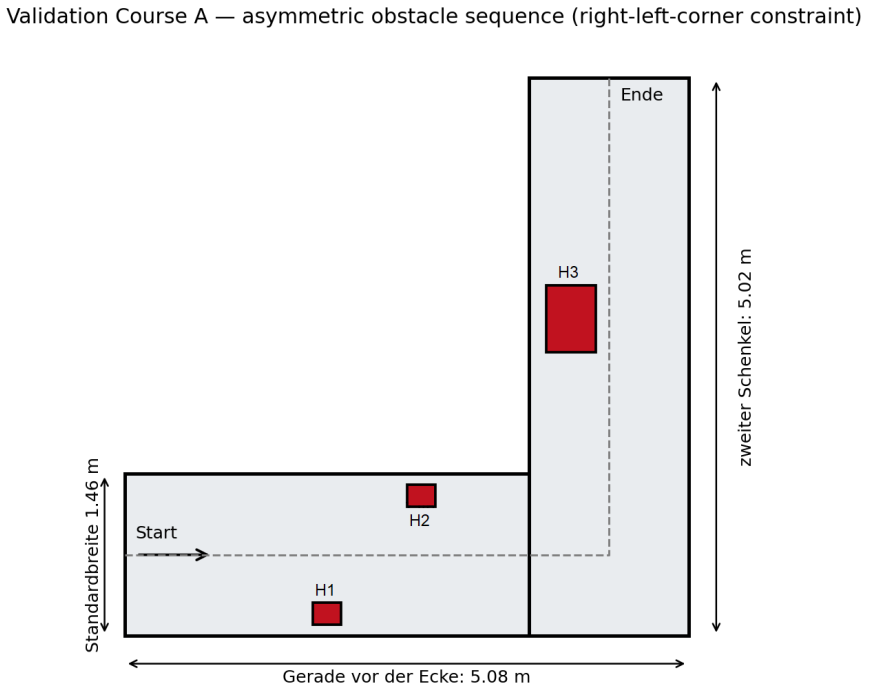

Abbildung 2.3: Versuchsaufbau Aufnahme 1: Korridor mit einem statischen Hindernis.

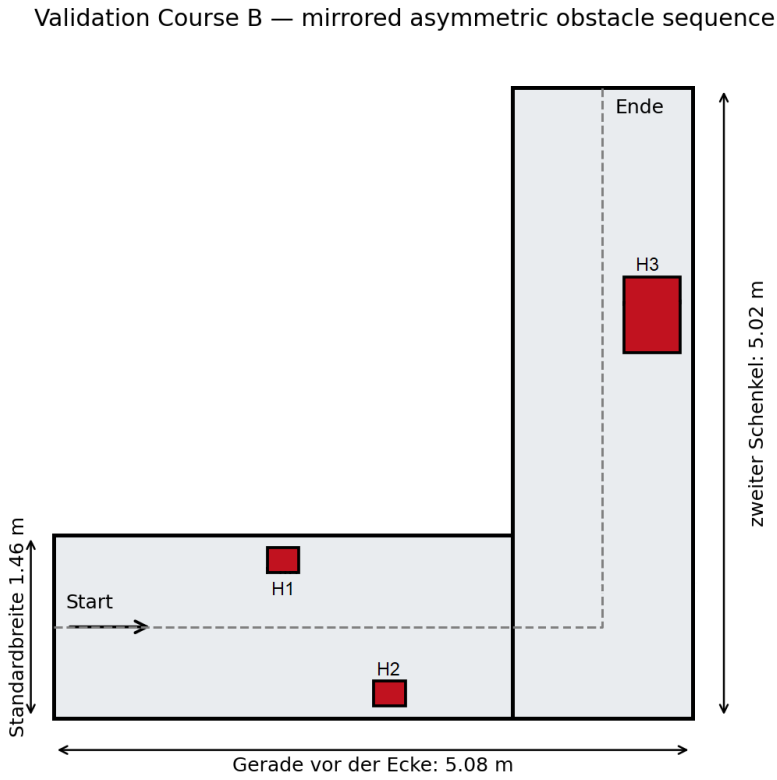

Abbildung 2.4: Versuchsaufbau Aufnahme 2: Mehrere Hindernisse und dynamische Veränderung der Umgebung.

2.7.2 Parameter

Repo 5 Konfiguration

2.7.3 Messmethodik

Zur Analyse wurden alle relevanten Systemgrößen aufgezeichnet und anschließend ausgewertet. Dazu gehören insbesondere:

- Distanzverlauf,
- Heading-Winkel,
- Geschwindigkeit,
- Gap-Erkennung.

Aufzeichnung :
in Repo 7.11 Bags
+ Abbildung von Foxglove

3 Diskussion und Bewertung

Dieses Kapitel bewertet die entwickelte Systemarchitektur hinsichtlich ihrer technischen Eignung, Robustheit und Übertragbarkeit. Dabei werden sowohl algorithmische als auch systemtechnische Aspekte analysiert.

3.1 Bewertung der gewählten Gesamtarchitektur

Die Entscheidung für eine strikt scan-basierte Pipeline stellt eine bewusste Abkehr von komplexeren, mehrstufigen Wahrnehmungsansätzen dar. Diese Reduktion des Datenpfads führt zu mehreren signifikanten Vorteilen.

3.1.1 Reduktion von Komplexität

Durch den direkten Übergang von `/scan` zu einer gefilterten Scan-Repräsentation wird auf zusätzliche Abstraktionsebenen wie Objektextraktion oder Hindernismodellierung verzichtet. Dies reduziert:

- die Anzahl potenzieller Fehlerquellen,
- den Parametrierungsaufwand,
- die Latenz innerhalb der Verarbeitungskette.

Gerade für reaktive Verfahren wie Follow-the-Gap ist diese Vereinfachung fachlich sinnvoll, da der Algorithmus explizit für rohnahe Sensordaten formuliert wurde [1].

3.1.2 Vergleich mit alternativen Ansätzen

Im Gegensatz zu globalen Planern oder kartengestützten Verfahren besitzt der implementierte Stack keine Kenntnis über die Umgebung außerhalb des aktuellen Sichtfelds. Dies führt zu folgenden Konsequenzen:

- hohe Reaktionsfähigkeit auf lokale Hindernisse,
- jedoch eingeschränkte Fähigkeit zur global optimalen Pfadplanung.

Die Architektur ist daher klar als **lokaler reaktiver Planer** einzuordnen und nicht als vollständiges Navigationssystem.

3.2 Analyse der Planner- und Control-Schicht

Die Trennung zwischen algorithmischem Kern, Planner und Control stellt eine klare funktionale Hierarchie dar.

Die gewählte Aufteilung ermöglicht:

- Wiederverwendbarkeit des FTG-Kerns,
- unabhängige Anpassung der Fahrstrategie,
- klare Definition von Verantwortlichkeiten innerhalb der Pipeline.

Insbesondere fungiert der Planner als **Policy-Schicht**, die sicherstellt, dass die reaktive Planung in physikalisch realisierbare und sichere Fahrbefehle überführt wird.

3.3 Bewertung der experimentellen Ergebnisse

Die durchgeführten Fahrversuche zeigen, dass das System in strukturierten Innenräumen grundsätzlich funktionsfähig ist, jedoch klare Abhängigkeiten von der Umgebungskonfiguration aufweist.

3.3.1 Quantitativer Vergleich der Validierungsläufe

Tabelle 3.1 fasst die wichtigsten Metriken der beiden Testläufe zusammen.

Tabelle 3.1: Vergleich der Metriken: Validierung Test 1 vs. Test 2

Metrik	**Test 1**	**Test 2**
Gap erkannt	90%	56%
Speed = 0 (Zeitanteil)	10%	44%
∅ Geschwindigkeit	0,37 m/s	0,40 m/s
Min. Clearance	0,27 m	0,34 m
Kollisionen	0	0
Heading-Bereich	± 0,8 rad	± 0,8 rad

3.3.2 Detailbewertung der Szenarien

- **Test 1:** Alle Hindernisse wurden umfahren und die 90°-Ecke gemeistert. Der kurze Stopp an der Ecke resultiert aus dem `kCarRadius`-Parameter von 0,4 m, der hier eine konservative Reaktion auslöste.

- **Test 2:** Dieser Lauf ist als eingeschränkt zu bewerten. Ein Zeitanteil von 44% mit Geschwindigkeit 0 sowie eine Gap-Erkennungsrate von nur 56% weisen auf Instabilitäten hin. Die Ursache liegt im 180° FOV, das zu breit gewählt war, wodurch Seitenwände die Lückenerkennung störten.

3.3.3 Empfehlungen aus der Validierung

Bewertung für alle Parameter. Basierend auf den Ergebnissen wird empfohlen, das FOV auf 120° zurückzusetzen, um Störeinflüsse durch Wände zu minimieren. Zudem sollte der `kCarRadius` von 0,4 m auf 0,20 m reduziert werden, um dem realen 30 cm breiten Fahrzeug besser gerecht zu werden.

3.3.4 Robustheit gegenüber statischen Hindernissen

In beiden Versuchsreihen konnte das Fahrzeug Hindernisse erkennen und geeignete Ausweichbewegungen berechnen. Die Anpassung der Geschwindigkeit in Abhängigkeit vom Frontabstand erwies sich als stabil und reproduzierbar.

3.3.5 Verhalten bei dynamischen Änderungen

Besonders relevant ist das Verhalten bei der Entfernung eines Hindernisses während der Fahrt. Hier zeigt sich ein charakteristischer Vorteil reaktiver Systeme:

- sofortige Anpassung an die neue Umgebung,
- keine Notwendigkeit zur Replanung einer globalen Route.

Dieses Verhalten bestätigt die Eignung des FTG-Ansatzes für dynamische Szenarien.

3.3.6 Grenzfälle

Trotz der positiven Ergebnisse treten typische Einschränkungen auf:

- keine strategische Entscheidung in Sackgassen,
- potenziell suboptimale Pfadwahl in komplexen Geometrien,
- Abhängigkeit von Sichtfeldbegrenzungen.

Diese Einschränkungen sind inhärent im Algorithmus begründet und nicht als Implementierungsfehler zu interpretieren.

3.4 Einordnung in den wissenschaftlichen Kontext

Die Kombination aus klassischem FTG-Ansatz [1], CTU-Referenzimplementierung [3] und moderner ROS 2-Infrastruktur [5] zeigt, wie etablierte Verfahren in aktuelle Softwarearchitekturen überführt werden können.

Die vorliegende Arbeit leistet dabei keinen Beitrag zur Weiterentwicklung des Algorithmus selbst, sondern zur:

- systematischen Integration,
- reproduzierbaren Umsetzung,
- technischen Konsolidierung eines bestehenden Ansatzes.

4 Fazit und Ausblick

4.1 Fazit

Im Rahmen dieser Projektarbeit wurde die vorhandene FTG-Dokumentation an den aktuellen technischen Stand des Repositories `ftg_mxck` und an die MXCK-Umgebung aus `mxck2_ws` angepasst. Der wesentliche inhaltliche Gewinn besteht darin, dass nun nicht mehr mehrere widersprüchliche Architekturbilder nebeneinanderstehen, sondern eine klar definierte Standardpipeline beschrieben wird.

Die überarbeitete Kernarchitektur lautet:

`/scan` → `scan_preprocessor_node` → `/autonomous/ftg/scan_filtered` → `follow_the_gap_v0` → `ftg_planner_node` → `ftg_command_node` → `/autonomous/ackermann_cmd`

Damit wurden folgende Ziele erreicht:

- Die Dokumentation entspricht der aktuellen scan-basierten Hauptpipeline des Repositories.
- Die Rollen von Perception, FTG-Kern, Planner und Control sind sauber getrennt.
- Die theoretische Erläuterung des FTG-Verfahrens wurde auf die Originalliteratur und die CTU/F1TENTH-Umsetzung zurückgeführt.
- Die ROS 2-spezifische Einordnung wurde durch die CTU-Arbeit zur ROS 2-Migration und Prozessanalyse gestärkt.
- Veraltete oder missverständliche Beschreibungen des Legacy-Pfads wurden aus dem Zentrum der Dokumentation entfernt.

Insgesamt ist die Arbeit damit nicht nur eine Beschreibung eines Algorithmus, sondern eine technische Bereinigung und Präzisierung eines gesamten ROS 2-Stacks für ein reales Lehr- und Entwicklungsfahrzeug.

4.2 Ausblick

Literaturverzeichnis

[1] Sezer, V.; Gokasan, M.: *A Novel Obstacle Avoidance Algorithm: "Follow the Gap Method"*. Robotics and Autonomous Systems, Vol. 60, No. 9, 2012, pp. 1123–1134. DOI: 10.1016/j.robot.2012.05.021.

[2] Endler, M.: *Using ROS 2 for High-Speed Maneuvering in Autonomous Driving*. Bachelor's Thesis, Czech Technical University in Prague, 2022.

[3] Czech Technical University – Intelligent and Interactive Graphics Group (CTU-IIG): *F1TENTH Follow-the-Gap Implementation (f1t-ftg)*. Available at: `https://github.com/CTU-IIG/f1t-ftg` (Accessed: 2026-04).

[4] O'Kelly, M. et al.: *F1TENTH: An Open-Source Evaluation Environment for Continuous Control and Reinforcement Learning*. NeurIPS Workshop on Machine Learning for Autonomous Driving, 2019.

[5] Open Robotics: *ROS 2 Documentation*. Available at: `https://docs.ros.org` (Accessed: 2026-04).

[6] Elalami, O. et al.: *ftg_mxck – ROS 2 Follow-the-Gap Stack for MXCarkit*. GitHub Repository. Available at: `https://github.com/OmarMElalami/ftg_mxck` (Accessed: 2026-04).

[7] MXCK Platform: *mxck2_ws – ROS 2 Workspace for MXCarkit*. Available at: `https://github.com/william-mx/mxck2_ws` (Accessed: 2026-04).